A design document for your Back End, giving the overall structure of your solution, showing the classes and methods in a UML class diagram and a brief (one sentence) description of the intention of each method and class in a table.

1. Design (Primarily in the design document)

Architecture
- clearly documents structure of solution
- explicitly describes intention of each class and method
- accurately reflects solution structure
- clearly shows where inputs and outputs fit in

Completeness
- evidently addresses all required functionality
  (solution has parts to address all required operations and results)
- has specified inputs, outputs and files (only!)

1. Purpose

The Back End is an overnight batch processor that reads the Old Master Bank Accounts File and the Merged Bank Account Transaction File produced by one or more Front End sessions. It applies all transactions, deducts per-transaction fees based on each account's payment plan (SP or NP), enforces business constraints, and produces the New Master Bank Accounts File and the New Current Bank Accounts File.

2. System Architecture

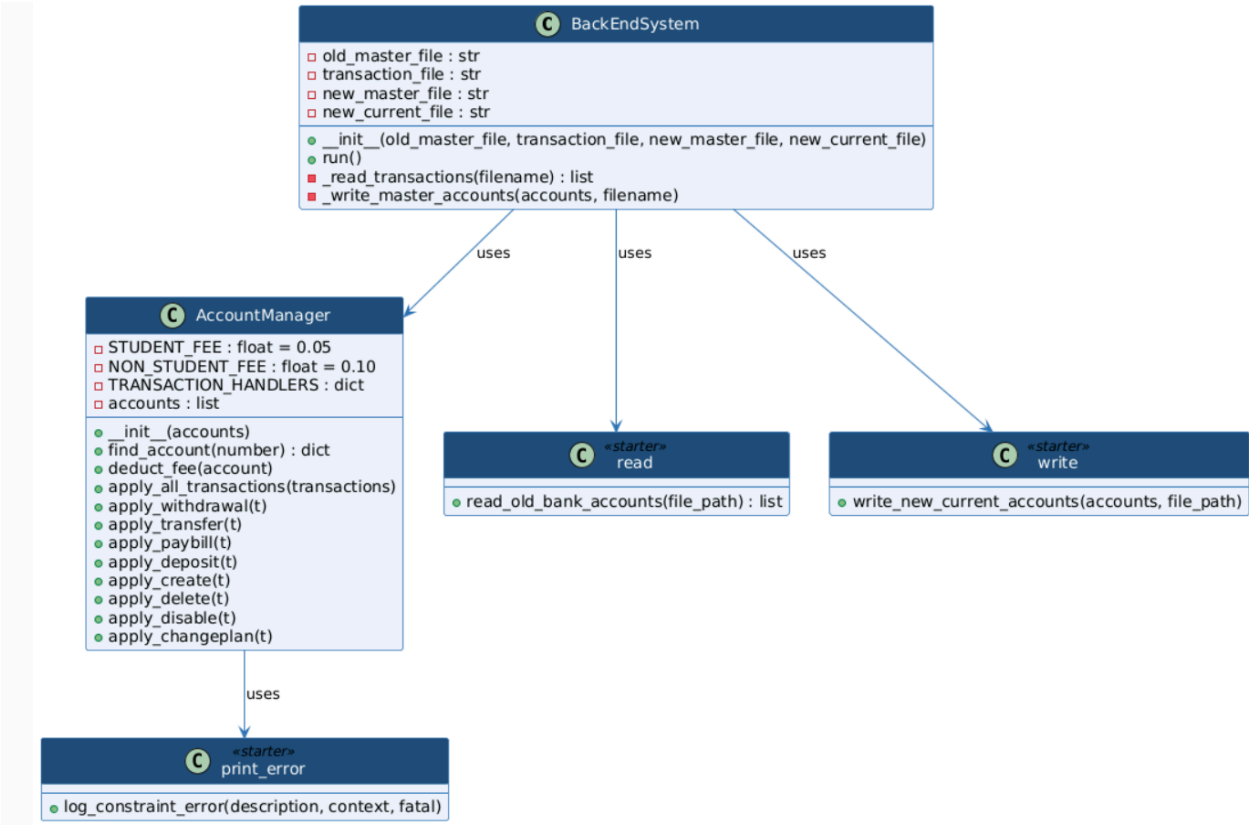
Class	File	Responsibility
BackEndSystem	back_end.py	Entry point. Coordinates the full pipeline from reading inputs to writing outputs.
AccountManager	account_manager.py	Applies transactions to accounts, enforces constraints, deducts fees, and logs errors.
read_old_bank_accounts	read.py ★	Reads and validates the Old Master Bank Accounts File into a list of account dicts.
write_new_current_accounts	write.py ★	Writes the New Current Bank Accounts File in 39-character fixed-width format.
log_constraint_error	print_error.py ★	Logs errors to the terminal in the required format; exits immediately on fatal errors.

★ = provided starter code module

<u>Data Flow</u>

Direction	File	Format	Description
Input	old_master.txt	42 chars/line	Previous day's master accounts with transaction counts
Input	merged_transactions.txt	40 chars/line	Concatenated Front End transaction files from the day
Output	new_master.txt	42 chars/line	Updated master accounts file, sorted by account number
Output	new_current.txt	37 chars/line	Simplified accounts file for tomorrow's Front End sessions

4. UML Class Diagram



5. Class and Method Descriptions

Class / Method	File	Description
BackEndSystem	back_end.py	Primary orchestrator that coordinates the full pipeline from reading inputs to writing outputs.
__init__		Stores the four file paths (old master, merged transactions, new master, new current) needed for the pipeline.
run		Executes the full pipeline: loads accounts, loads transactions, applies transactions, sorts accounts, writes both output files.
FileHandler	file_handler.py	Handles all fixed-width file reading and writing for the Back End system.
read_master_accounts		Reads the Old Master Bank Accounts File and returns a list of account dictionaries; exits on fatal file errors.
read_transactions		Reads the Merged Bank Account Transaction File and returns a list of transaction dictionaries.
write_master_accounts		Writes the New Master Bank Accounts File in 42-character fixed-width format, sorted by account number.
write_current_accounts		Writes the New Current Bank Accounts File in 37-character fixed-width format for the Front End to consume.
_format_master_line		Formats a single account dictionary into a 42-character master file line.
_format_current_line		Formats a single account dictionary into a 37-character current accounts file line.
AccountManager	account_manager.py	Applies all banking transactions to the account list, enforces constraints, and logs all errors to the terminal.
__init__		Stores the accounts list that will be modified during transaction processing.
find_account		Returns the account dictionary matching the given account number, or None if not found.
log_error		Prints a constraint violation or fatal error to the terminal in the required ERROR: <msg> format.
deduct_fee		Deducts the per-transaction fee (SP: $0.05, NP: $0.10) from the account and increments its transaction counter.
apply_all_transactions		Iterates through the full transaction list and routes each transaction to its handler method by code.

apply_withdrawal		Deducts the withdrawal amount from the account; logs an error if balance would go negative.
apply_transfer		Deducts the transfer amount from the source account; logs an error if balance would go negative.
apply_paybill		Deducts the bill payment amount from the account; logs an error if balance would go negative.
apply_deposit		Adds the deposited amount to the account balance.
apply_create		Creates a new account and adds it to the accounts list; logs an error if the account number already exists.
apply_delete		Removes the specified account from the accounts list; logs an error if the account is not found.
apply_disable		Sets the account status to Disabled (D); logs an error if the account is not found.
apply_changeplan		Toggles the account plan between SP (student) and NP (non-student); logs an error if account is not found.
read_old_bank_accounts	read.py ★	Reads and validates the Old Master Bank Accounts File (45 chars/line) and returns a list of account dicts; logs fatal errors for malformed lines.
write_new_current_accounts	write.py ★	Writes the New Current Bank Accounts File (39 chars/line) with validation, including the plan field, for the Front End to consume.
log_constraint_error	print_error.py ★	Logs errors in the required ERROR: format; exits the program immediately if the error is marked as fatal.

6. Rules and constraints

Rule	**Behaviour**
Negative balance	Transaction is rejected and an ERROR is logged to the terminal. Processing continues.
Duplicate account number	Create transaction is rejected and an ERROR is logged. Processing continues.
Student plan fee (SP)	$0.05 is deducted from the account balance per transaction.
Non-student plan fee (NP)	$0.10 is deducted from the account balance per transaction.
Fatal file error	An ERROR is logged to the terminal and the program exits immediately.
Output sort order	New master file is always written in ascending order by account number.